A ARKA-Lite

EQUIPO ARKA-LITOS · UPSKILLING BACKEND JAVA

Del endpoint al clúster

Cómo construimos dos microservicios con sus datos, su plataforma y su operación.

2 microservicios	**2** bases PostgreSQL	**1** comando levanta todo	**0** eventos perdidos en caídas

STAFF DNTIC



Qué vamos a ver

Siete partes, en el orden en que construimos el sistema; nos detenemos en las tres del centro.



1 Fundamentos

VISTA GENERAL

Analizar primero · Código mantenible · Reglas con su prueba

2 Arquitectura

VISTA GENERAL

Las capas del sistema · Cómo se conectan · Hexagonal por dentro

3 Base de datos

EN DETALLE

Modelo de datos · Qué es un ORM · Datos de conexión

4 Interconexión de servicios

EN DETALLE

Síncrono y asíncrono · Patrón Outbox · Idempotencia

5 DevOps: Docker y Kubernetes

EN DETALLE

Docker · CI/CD · Kubernetes

6 Observabilidad

VISTA GENERAL

¿Está vivo? · ¿Qué pasó? · ¿Por dónde pasó?

7 Demostración

EN VIDEO

Crear y enviar · Una caída grabada · Reentregar

Lo primero: analizar antes de programar

Tres preguntas antes de escribir código; sus respuestas se volvieron el diseño del sistema.

LO QUE PREGUNTAMOS	LO QUE RESPONDIÓ EL NEGOCIO	EN QUÉ SE CONVIRTIÓ
¿Qué tiene que pasar?	Se registra, se envía, se avisa y se decide	Cuatro estados: borrador, enviada, aprobada y rechazada
¿Quién hace cada paso?	Quien solicita envía, el sistema avisa y otra persona decide	Dos servicios: uno para las solicitudes y otro para los avisos
¿Qué no se puede permitir?	Las reglas: se envía solo un borrador, una vez; se decide solo sobre una enviada	Cada regla con su prueba y un error que explica por qué

Código mantenible: buenas prácticas

Seis prácticas que hacen que un cambio toque un solo lugar, en cualquier lenguaje.

Una tarea por pieza

Quien recibe el pedido no decide; quien decide no guarda.

Piezas que se conectan

Cada pieza se puede cambiar o probar por separado.

Reglas en un solo lugar

No repetidas en la pantalla y en la base.

Errores claros

Siempre igual, con un mensaje que explica qué pasó.

Nombres del negocio

Solicitud, enviar, aprobar: se lee como el proceso.

Cada regla con su prueba

Una prueba automática avisa si algo se rompió.

Fundamentos: lo que nos quedó

LO QUE APRENDIMOS

3

preguntas antes de programar

6

prácticas que sirven en cualquier lenguaje

Analizar antes de programar

Qué tiene que pasar, quién lo hace y qué no se permite.

VALOR

Reglas claras antes de elegir la tecnología.

Separar responsabilidades paga pronto

Pantalla, reglas y datos en piezas distintas.

VALOR

Un cambio de regla toca un solo lugar.

Las reglas no dependen de la herramienta

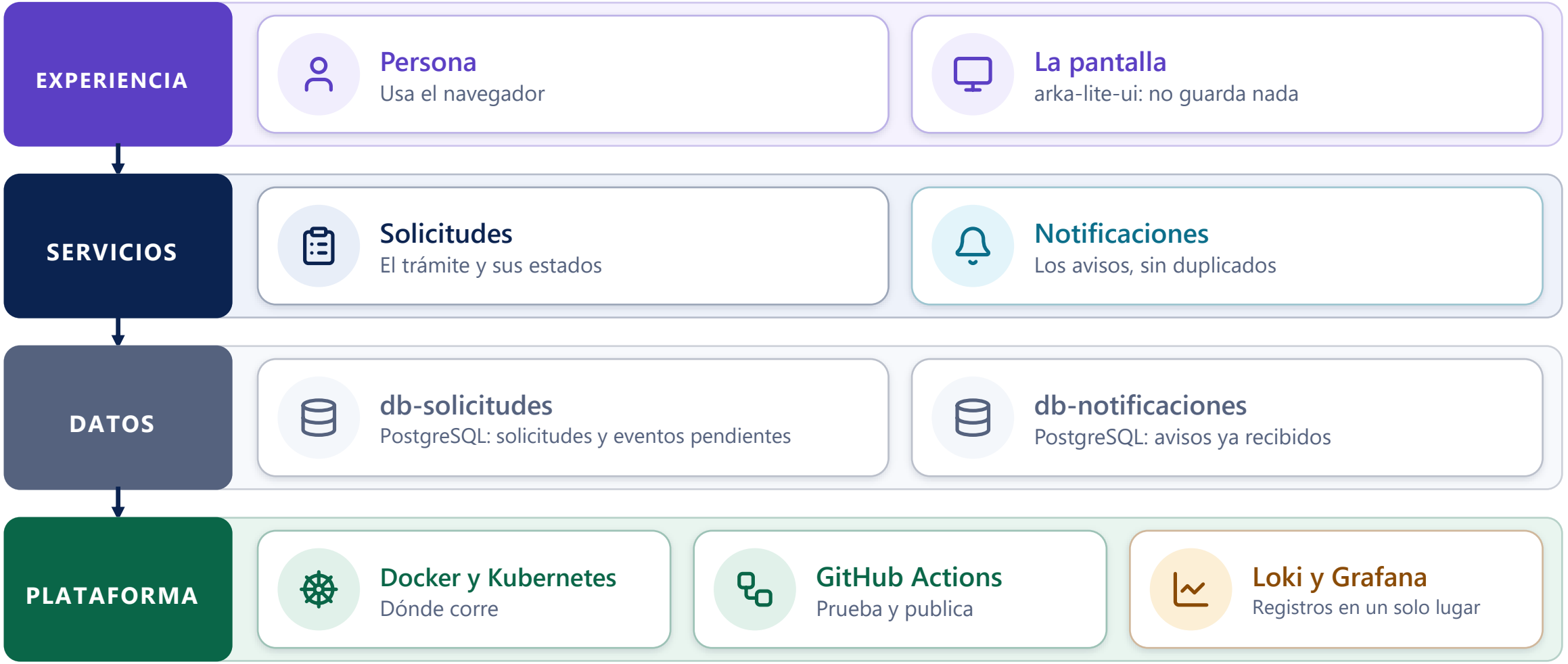
El negocio no sabe si detrás hay una base, una web o un framework.

VALOR

Se prueba rápido y sobrevive a un cambio de tecnología.

El sistema de un vistazo

Cada capa tiene una sola responsabilidad.



Cómo funciona en la práctica

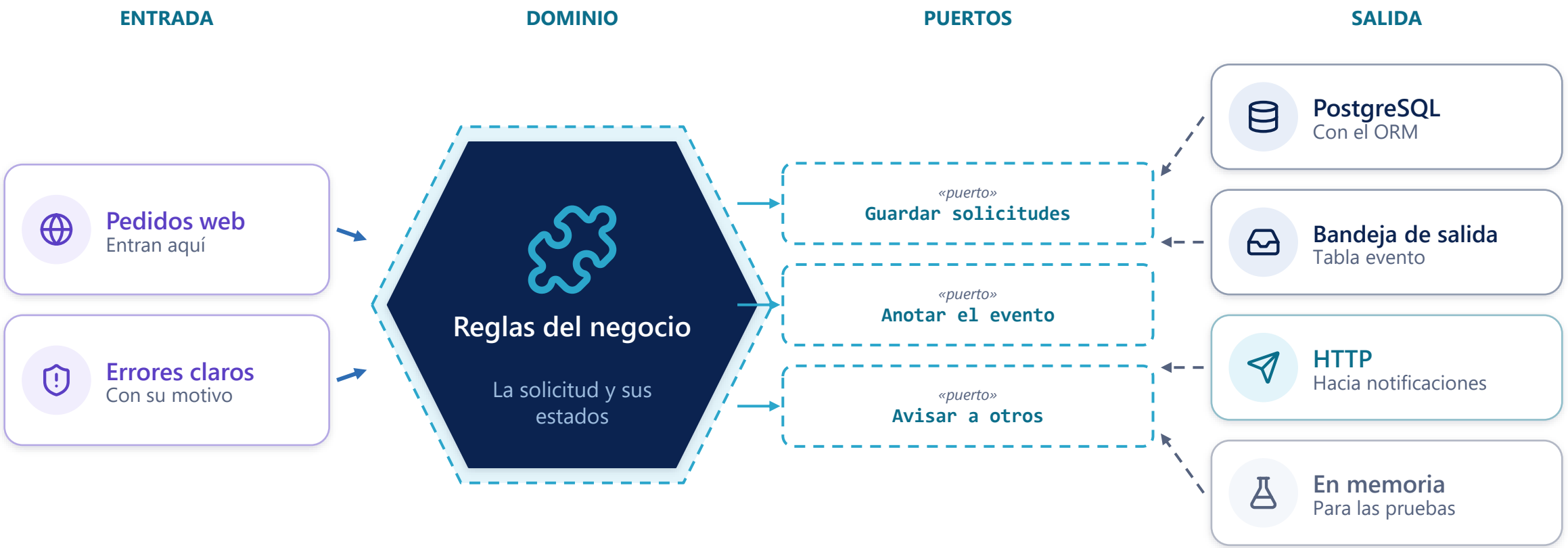
Seis conexiones, y ninguna une las dos bases.



Ninguna flecha une las dos bases: cada servicio es dueño de sus datos.

Por dentro de cada servicio: hexagonal

Una forma de ordenar el código por dentro; no cambia cómo se instala.



Piénsenlo así: Como un enchufe: el aparato funciona con cualquier toma que cumpla la norma, sin saber de qué planta viene la electricidad.

Arquitectura: lo que nos quedó



LO QUE APRENDIMOS

2

servicios, cada uno con su base

0

tablas compartidas entre servicios



Cada servicio, dueño de sus datos

Nadie lee ni escribe en la base del otro.

VALOR

Cada uno cambia sin romper al otro.



Entre servicios viajan mensajes

Un evento pequeño con su número.

VALOR

Un acuerdo claro y fácil de revisar.



Hexagonal ordena por dentro

No cambia la instalación: un contenedor por servicio.

VALOR

Cambiar la base o la web sin tocar las reglas.



Separar tiene un costo

Más red, dos bases y fallas parciales.

VALOR

Separar solo cuando el negocio lo pide.

Dos bases, una por servicio

Cada microservicio tiene su propia base, y las dos corren en PostgreSQL.

2

bases de datos

db-solicitudes y db-notificaciones: una por servicio.

17

PostgreSQL

El mismo motor y la misma versión en las dos.

0

JOIN entre bases

Ni consultas ni llaves foráneas que crucen de una a otra.

1

disco por base

Los datos no se pierden aunque el servicio se reinicie.

La estructura de las bases

Tres tablas en dos bases, y una sola cosa cruza la frontera: el número del evento.

db-solicitudes
PostgreSQL 17

solicitud

- # **id** llave primaria
- # **tipo**
- # **estado** solo 4 valores

evento

- # **id** llave primaria
- # **solicitud_id** llave foránea
- # **evento_id** número único
- # **procesado** ¿ya se entregó?

evento_id, por la red →

sin llaves foráneas entre bases

db-notificaciones
PostgreSQL 17

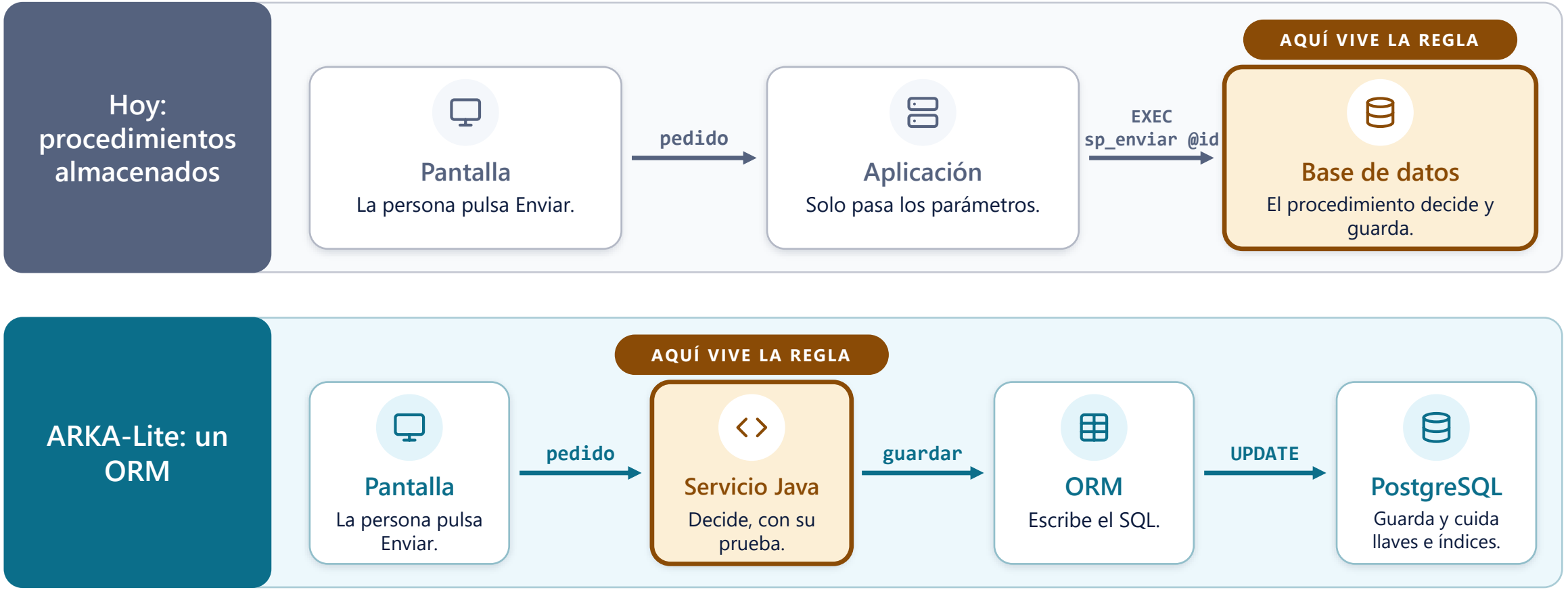
evento_procesado

- # **evento_id** llave primaria
- # **uno por evento recibido**

¿Por qué importa? Cada servicio cambia su estructura sin coordinarse con el otro; solo el número del evento es compartido.

Del procedimiento almacenado al ORM

La diferencia no es la base de datos: es dónde vive la regla del negocio.



¿Por qué importa? No es todo o nada: las reglas van al código y los reportes o consultas pesadas pueden seguir en SQL.

La misma regla, escrita de las dos formas

Regla del negocio: solo se puede enviar una solicitud que está en borrador.

Hoy: en un procedimiento

```
CREATE PROCEDURE sp_enviar @id VARCHAR(20) AS
BEGIN
  IF (SELECT estado FROM solicitud
      WHERE id = @id) <> 'BORRADOR'
    THROW 50001, 'Solo se envía en BORRADOR', 1;
  UPDATE solicitud SET estado = 'ENVIADA'
   WHERE id = @id;
END
```

- La regla y el dato viven juntos en la base
- Probarla exige una base con datos
- Cambiarla es instalar en la base

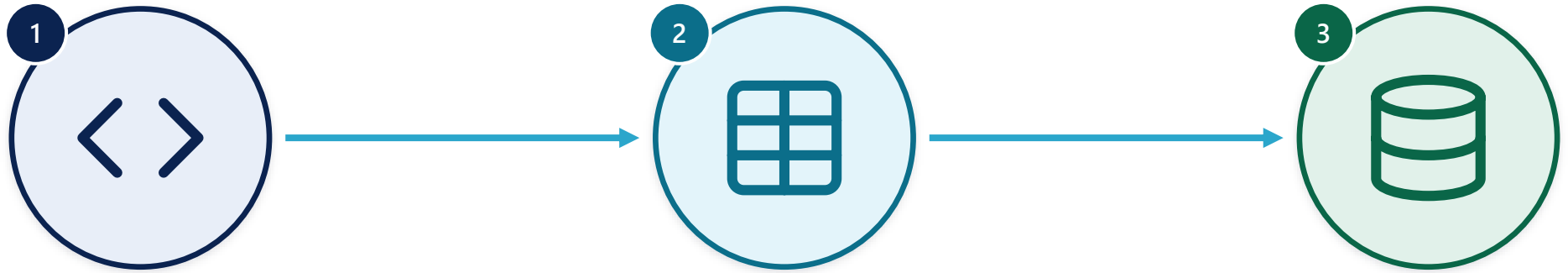
ARKA-Lite: en el código

```
public Solicitud enviar() {
    if (estado != Estado.BORRADOR)
        throw new TransicionNoPermitida(
            "solo se puede enviar en BORRADOR");
    return new Solicitud(id, tipo, Estado.ENVIADA);
}
// El ORM escribe el SQL:
// UPDATE solicitud SET estado = ? WHERE id = ?
```

- La regla vive en el código, con su prueba
- Se prueba en milisegundos, sin base
- La base sigue cuidando llaves e índices

¿Qué es un ORM?

Un traductor: la aplicación habla en objetos, la base habla en tablas, y el ORM traduce entre los dos.



La aplicación

Dice: guarda esta solicitud.

El ORM

Traduce y escribe el SQL. Aquí: Hibernate.

La base

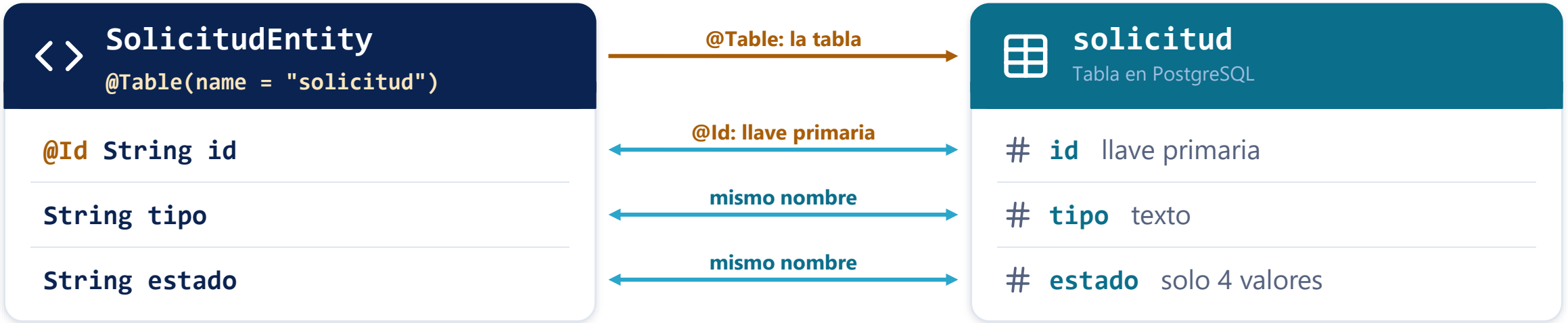
Recibe el SQL y guarda la fila.

Piénsenlo así: Como un intérprete en una reunión: cada quien habla su idioma y el intérprete traduce. Ninguno de los dos tiene que aprender el idioma del otro.

¿Por qué importa? Menos SQL repetitivo y siempre con parámetros; lo que importa de la base lo seguimos diseñando nosotros.

El ORM por dentro: el mapa entre la clase y la tabla

Unas etiquetas en el código forman el mapa entre la clase y la tabla; con ese mapa, el ORM escribe el SQL.



CON ESE MAPA, EL ORM ESCRIBE EL SQL QUE LLEGA A LA BASE

`cambiar el estado` →

```
UPDATE solicitud SET tipo = ?, estado = ? WHERE id = ?
```

Piénsenlo así: Como el diccionario de un intérprete: las etiquetas le dicen qué parte del código corresponde a qué tabla y a qué columna.

¿Dónde guardamos los datos de conexión?

La misma idea que ya conocemos: la dirección y la clave no van escritas en el código.

Hoy: web.config

- Un archivo en el servidor
- Guarda servidor, base, usuario y clave
- Cada servidor tiene su propia copia

VS

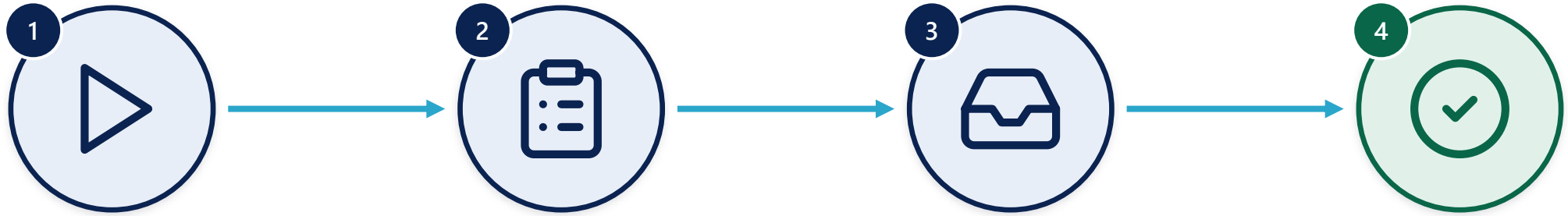
ARKA-Lite: variables de entorno

- El ambiente se las entrega a la aplicación al arrancar
- Guardan los mismos datos
- La misma aplicación sirve en todos los ambientes

¿Por qué importa? Cambiar de servidor o de clave no obliga a tocar el código ni a volver a construir la aplicación.

Una transacción: todo o nada

El BEGIN y COMMIT de siempre: la solicitud y su evento se guardan juntos o no se guarda nada.



BEGIN

Empieza la transacción.

UPDATE

La solicitud pasa a enviada.

INSERT

El evento queda pendiente.

COMMIT

Las dos cosas quedan guardadas.

si algo falla

ROLLBACK

No queda nada a medias: ni la solicitud ni el evento.

Piénsenlo así: Como una transferencia bancaria: o sale de una cuenta y entra en la otra, o no pasa nada. Nunca a medias.

Base de datos: lo que aprendimos

LO QUE APRENDIMOS

3

tablas en dos bases

0

consultas que crucen de una base a otra

El ORM no elimina el trabajo de base de datos

Escribe el SQL de rutina; índices, llaves y transacciones siguen siendo diseño.

VALOR

Reglas probadas en código y una base que se cuida.

La conexión no vive en el código

La entrega el ambiente donde corre la aplicación.

VALOR

Cambiar de servidor o de clave sin tocar el código.

La transacción sigue siendo la más confiable

Lo que cambia junto se guarda en el mismo COMMIT.

VALOR

Nunca quedan datos a medias.

La estructura también se controla

Se guarda y se revisa junto con el código.

VALOR

Instalaciones repetibles y errores visibles al inicio.

Síncrono o asíncrono

Si necesitamos el resultado para continuar, esperamos; si no, el trabajo se completa después.

Síncrono

- La pantalla llama y espera el resultado
- Respuesta inmediata
- Si el servicio se cae, la persona ve el error

vs

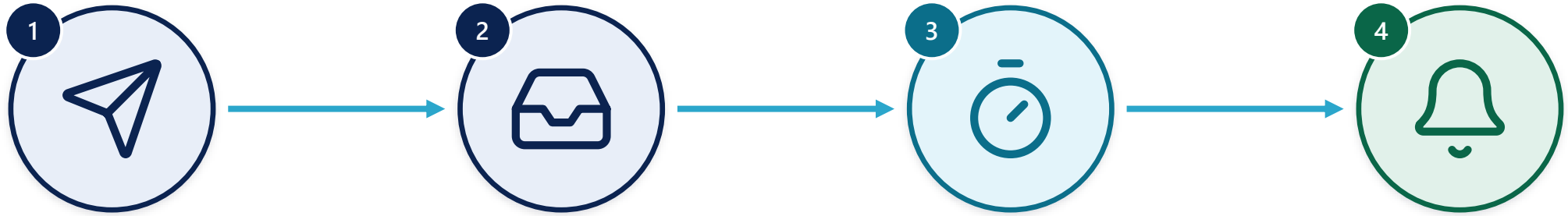
Asíncrono

- Solicitudes envía el evento y sigue con su trabajo
- La respuesta puede demorar unos segundos
- Si notificaciones se cae, el evento espera

Piénsenlo así: Síncrono es una llamada: me quedo en la línea hasta que me responden. Asíncrono es un correo: lo envío y sigo con lo mío.

Patrón Outbox: guardar primero, avisar después

La solicitud y su evento se guardan en la misma transacción; un proceso lo entrega después.



Enviar

Solicitud y evento, en una transacción.

Bandeja de salida

El evento espera en su tabla.

Relevador

Cada 5 segundos entrega los pendientes.

Notificaciones

Lo recibe y crea el aviso.

si falla

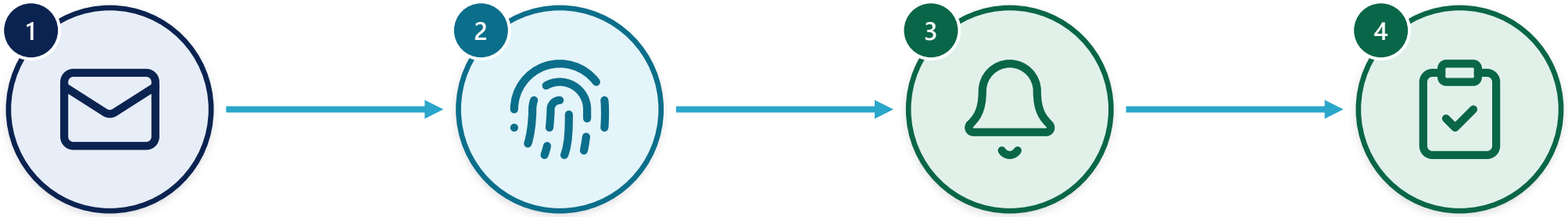
Sigue pendiente

Con el mismo número; se reintenta en la siguiente vuelta.

Piénsenlo así: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

Idempotencia: el mismo evento, un solo aviso

Repetir la entrega no multiplica el efecto.



Llega el evento

Trae su número.

¿Ya lo recibimos?

Se busca en la lista.

Crear el aviso

Solo si el número es nuevo.

Anotar el número

En la misma transacción.

ya estaba

Ignorarlo

No se crea otro aviso. Duplicados: cero.

Piénsenlo así: Como un portero con lista de invitados: si alguien ya entró, no lo vuelve a anotar.

Resumen: quién habla con quién, y cómo

El recorrido completo de una solicitud: una parte mientras la persona espera y otra después, por su cuenta.



¿Por qué importa? Las piezas se encuentran por su nombre y un número de seguimiento viaja en cada llamada: una sola búsqueda sigue todo el recorrido.

Interconexión: lo que aprendimos



LO QUE APRENDIMOS

0

eventos perdidos

0

avisos duplicados al reentregar



Esperar solo cuando hace falta

La pantalla espera la respuesta; el evento se entrega después.

VALOR

Una caída parcial no detiene a quien usa el sistema.



Guardar primero, avisar después

Solicitud y evento en la misma transacción.

VALOR

Un servicio caído no hace perder información.



Todo reintento necesita idempotencia

Un número fijo y la lista de los ya recibidos.

VALOR

Reintentar sin miedo a duplicar.



El formato es lo único compartido

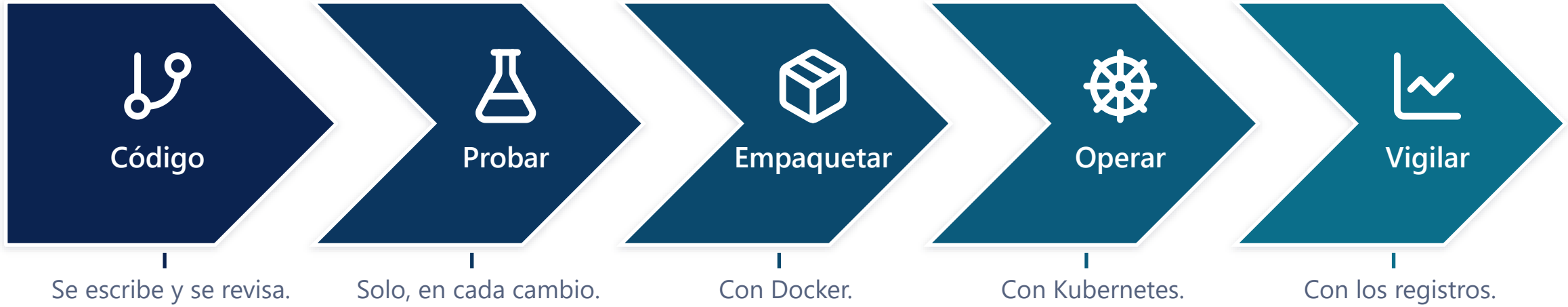
El mensaje tiene los mismos campos en los dos lados.

VALOR

Equipos independientes con un acuerdo claro.

¿Qué es DevOps? Del código a la operación

No es una herramienta ni un puesto: es un solo camino, casi todo automático, del código a la operación y de vuelta.



Piénsenlo así: Como una línea de producción: cada producto pasa por las mismas estaciones, en el mismo orden, y nadie se salta un paso.

Docker y Kubernetes: no compiten, se complementan

Docker construye la caja; Kubernetes la pone en ejecución y cuida su funcionamiento.

Docker: empaqueta

- Encapsula la app con todo lo que usa
- Crea una imagen que no cambia
- Se ejecuta igual en donde sea

+

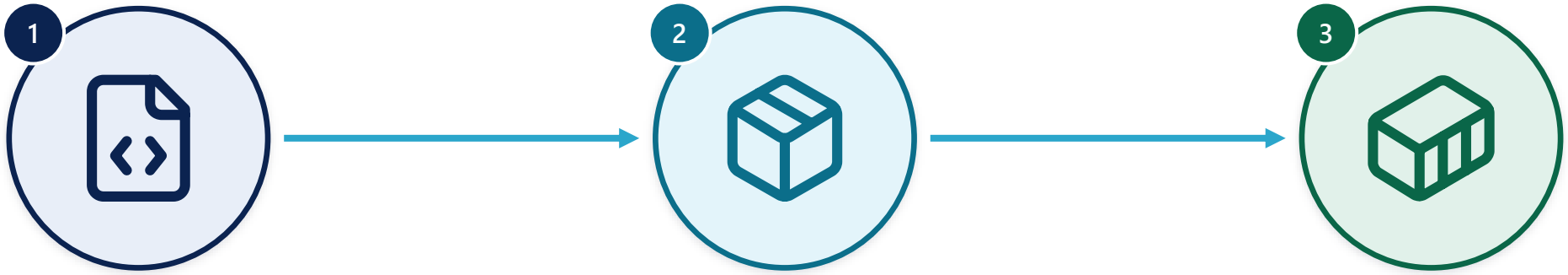
Kubernetes: administra

- Decide dónde corre cada contenedor
- Si uno se cae, levanta otro
- Puede correr más copias si hay carga

Piénsenlo así: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

De código a contenedor

Una receta, un molde y lo que sale del molde: la misma aplicación en cualquier computadora.



Dockerfile

Qué instalar y qué copiar.

Imagen

Solo la aplicación y los componentes que necesita.

Contenedor

La imagen en ejecución, con un usuario sin privilegios por seguridad.

Piénsenlo así: Como hornear: la receta es el Dockerfile, el molde es la imagen y cada pan que sale es un contenedor. Del mismo molde salen todos iguales.

Ventajas de Docker

Seis cosas que ganamos al empaquetar la aplicación en una imagen.

Igual en todos lados

La misma imagen en la laptop y en el servidor.

Liviano y rápido

Arranca en segundos, sin un sistema operativo completo.

Aislado

Cada aplicación con lo suyo: no interfieren entre sí.

Memoria y CPU con límite

Se puede poner un tope por contenedor.

Imágenes con versión

Cada versión se guarda; volver atrás es usar la anterior.

Un comando

Se levanta igual, sin una lista de pasos a mano.

Todo el sistema con un comando

Docker Compose levanta los ocho contenedores juntos, en orden y conectados.

```
>_ $ docker compose up --build -d
```

La pantalla

arka-lite-ui

Solicitudes

El trámite

Notificaciones

Los avisos

db-solicitudes

PostgreSQL

db-notificaciones

PostgreSQL

Promtail

Recolecta los registros

Loki

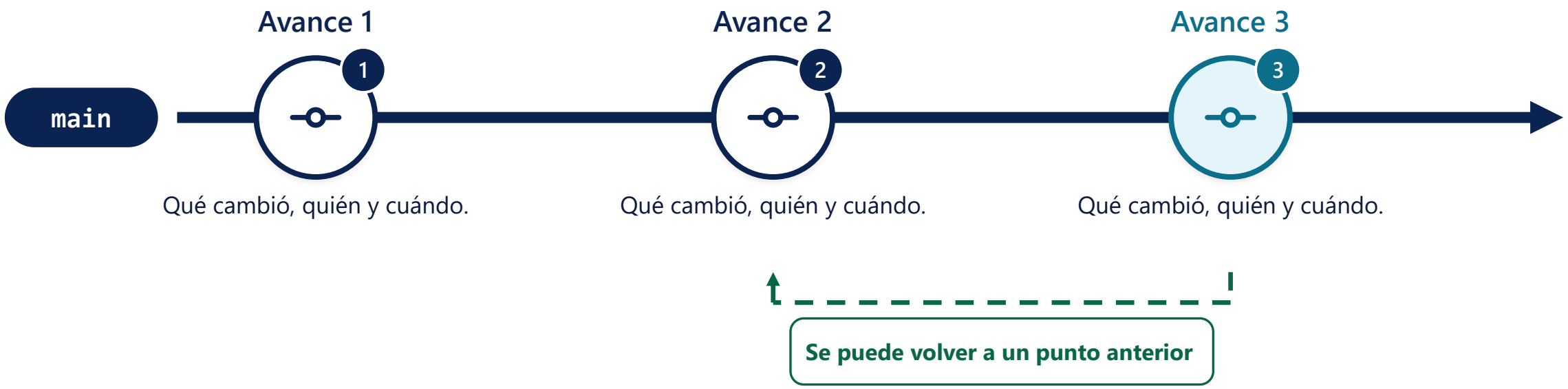
La base de los registros

Grafana

La pantalla para consultarlos

Git: el historial y main

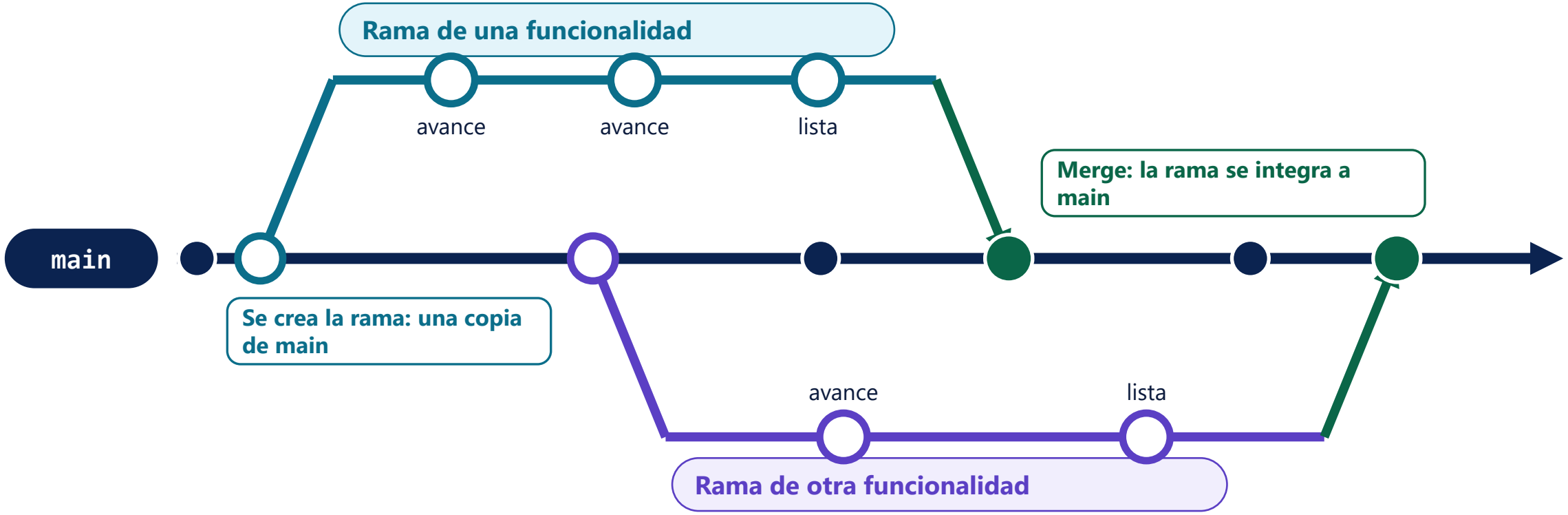
Git guarda cada avance como un punto en una línea de tiempo; la línea principal y estable se llama main.



Piénsenlo así: Como el historial de versiones de un documento compartido: se ve cada cambio y se puede recuperar uno anterior.

Las ramas: trabajar en paralelo sin tocar main

main no se edita directamente: cada funcionalidad se trabaja en su rama y luego se integra a main.



Piénsenlo así: Como trabajar en una copia del documento y, al final, integrar los cambios al original.

Cómo entra un cambio, paso por paso

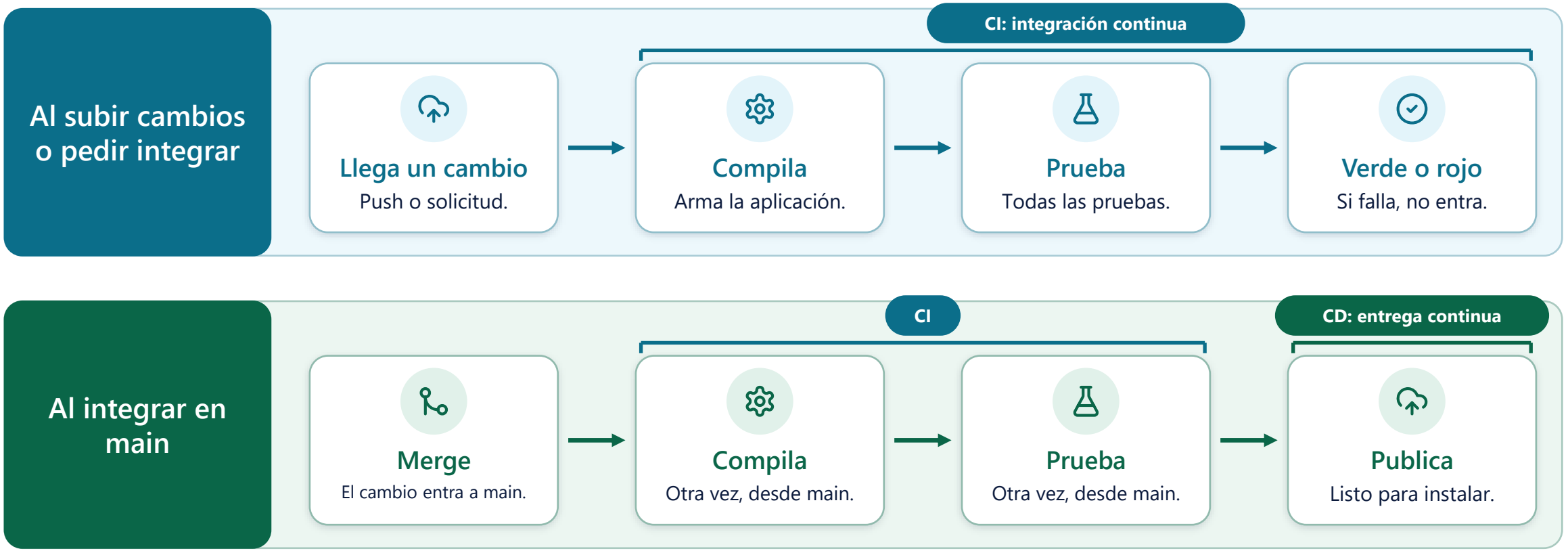
Cada cambio se trabaja en una rama y solo entra a main cuando otra persona lo revisó.



¿Por qué importa? Nada llega a la versión oficial sin que otra persona lo revise.

CI/CD: el control de calidad automático

El pipeline es una lista de pasos que corre sola cuando ocurre algo en el repositorio.



Piénsenlo así: Como el control de calidad de una fábrica: cada producto pasa las mismas revisiones, y el que falla no sale al mercado.

El pipeline se escribe como código

Cada herramienta guarda el pipeline en un archivo; todas dicen cuándo, dónde y qué pasos.

GitHub

```yaml
# GitHub · .github/workflows/ci.yml
on:                          # cuándo
  pull_request:
  push:
    branches: [ main ]
jobs:
  probar:
    runs-on: ubuntu-latest   # dónde
    steps:                   # qué pasos
      - run: mvn -B verify
```

- on: cuándo corre
- runs-on: dónde corre
- steps: qué pasos hace

GitLab

```yaml
# GitLab · .gitlab-ci.yml
probar:
  rules:                     # cuándo
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"
  image: maven:3             # dónde
  script:                    # qué pasos
    - mvn -B verify
```

- rules: cuándo corre
- image: dónde corre
- script: qué pasos hace

¿Por qué importa? Azure DevOps (azure-pipelines.yml) y Jenkins (Jenkinsfile) siguen la misma idea, cada uno con su archivo.

Qué se puede automatizar

Cada paso del pipeline es una tarea que deja de depender de alguien; en verde, los que ya hace ARKA-Lite.

Compilar

Que el código arme sin errores.

Pruebas unitarias

Que las reglas sigan funcionando.

Verificar el arranque

Que la aplicación levante con su base.

Publicar el paquete

La imagen, lista para instalar.

Calidad del código

Estilo, duplicados y complejidad.

Seguridad

Librerías con fallas o claves escritas en el código.

Instalar en pruebas

Un ambiente para revisar antes de producción.

Avisar al equipo

Un mensaje si algo falla.

Ejemplo: del push al despliegue



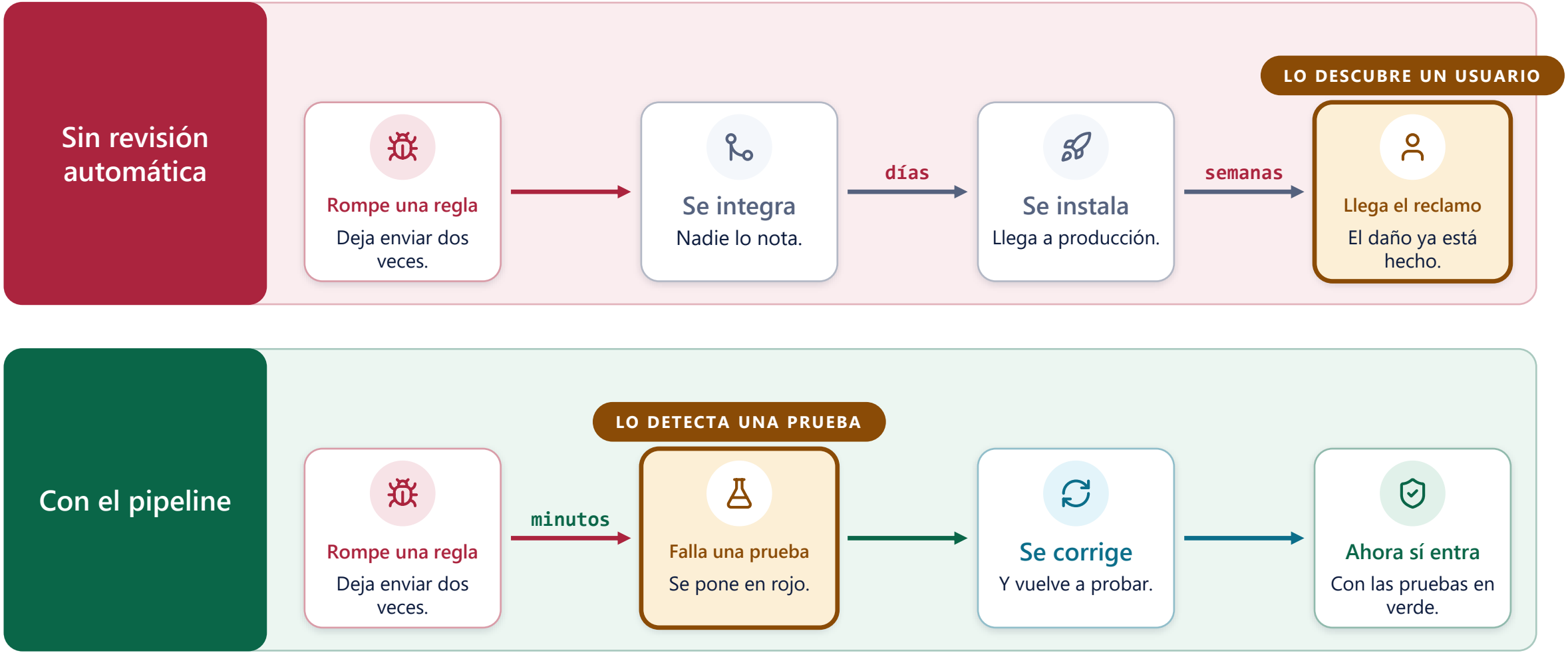
QUÉ VAN A VER

1. Quien desarrolla termina su cambio y hace push: lo sube a su rama en GitHub
2. Abre un Pull Request y otra persona revisa y aprueba el cambio
3. El pipeline arranca solo: compila y corre todas las pruebas
4. Con el merge, el cambio entra a main y se publica la imagen
5. Se instala la imagen nueva y los usuarios ya ven la versión 1.5

▶ Arranca solo · 0:47 · Alt+P pausa

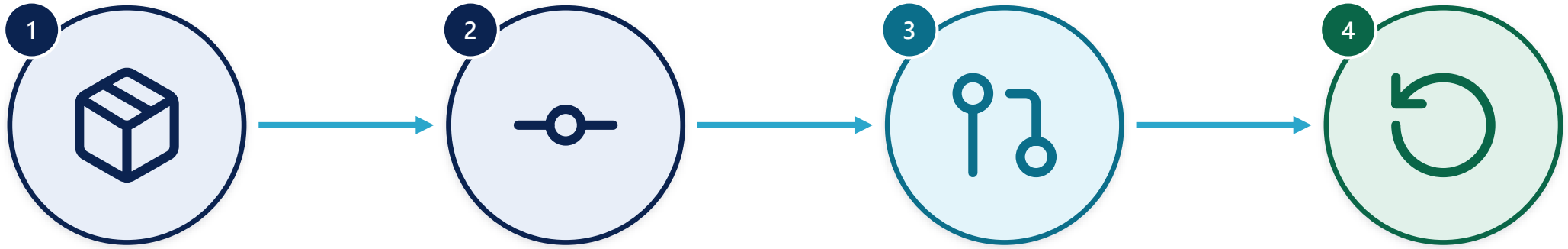
Cuando un cambio rompe algo

La misma falla, con y sin pipeline: la diferencia es cuándo se descubre.



Cada imagen, con su etiqueta

Cada imagen lleva el código del cambio que la generó: sabemos qué corre y podemos volver atrás.



La imagen

Lleva un código único.

El cambio

Qué líneas cambiaron.

La revisión

Quién lo pidió y quién lo revisó.

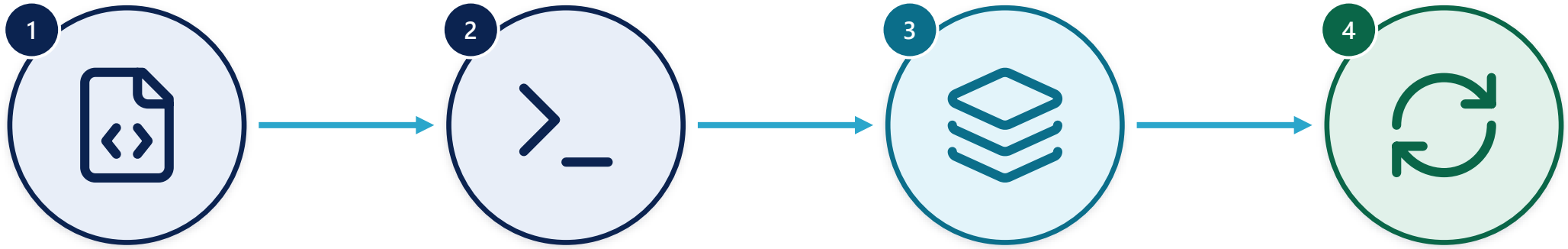
Volver atrás

Instalar la imagen anterior.

Piénsenlo así: Como el número de lote de un medicamento: si algo sale mal, se sabe exactamente de dónde vino.

Cómo trabaja Kubernetes

Le decimos cómo queremos el sistema; Kubernetes hace lo necesario para que así sea, todo el tiempo.



Un archivo

Qué imagen, cuántas copias y qué configuración.

Un comando

Se lo entrega a Kubernetes.

Lo crea

Levanta lo que pedimos.

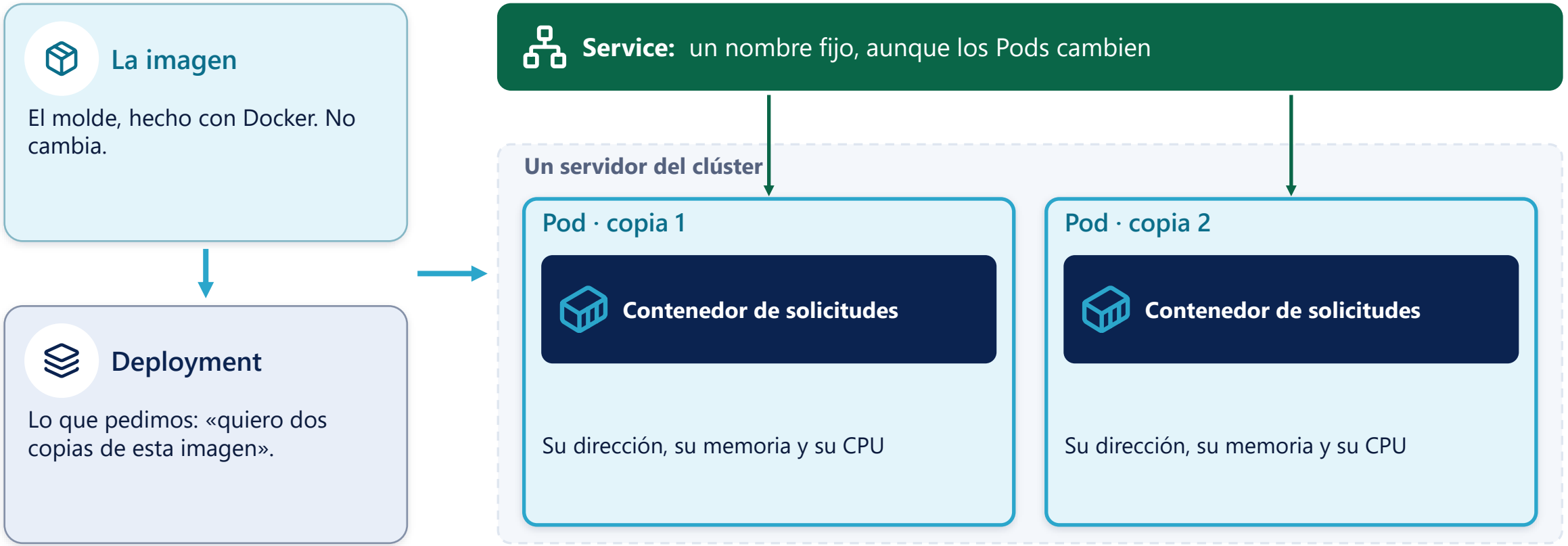
Lo mantiene

Compara lo que hay con lo pedido y corrige.

Piénsenlo así: Como un termostato: no le decimos cuándo prender el aire, le decimos la temperatura que queremos y él se encarga.

Contenedores y Pods: cómo se complementan

El contenedor es la aplicación corriendo; el Pod es el lugar que Kubernetes le da para vivir.



Piénsenlo así: Siguiendo con el puerto: el contenedor es la carga; el Pod es su espacio en el patio, con número y conexión. Si el contenedor se daña, el puerto trae otro.

Las bondades de Kubernetes

Lo que Docker empaqueta, Kubernetes lo mantiene funcionando.

Se repara solo

Si un Pod se cae, levanta otro con la misma imagen.

Crece con más copias

Más carga, más copias: se cambia un número.

Reparte el trabajo

El Service distribuye los pedidos entre las copias.

Actualiza sin apagar

Cambia las copias poco a poco; se puede volver atrás.

Memoria y CPU

A cada Pod, una reserva y un tope.

Datos y claves aparte

Las claves en un Secret; los datos en su propio disco.

Cuando un Pod se cae

Kubernetes vuelve solo al estado que le pedimos.

QUÉ FALLA

Se cae el servicio de solicitudes Por un error o porque se reinició el servidor.



RESULTADO

Nadie intervino Y los eventos pendientes siguen guardados en la base.

DevOps: lo que aprendimos

LO QUE APRENDIMOS

1

comando para levantar todo

1

imagen por cambio, rastreable

Docker y Kubernetes se complementan

Uno empaqueta; el otro ejecuta y cuida.

VALOR

Un mismo paquete, de la laptop al clúster.

La imagen es el entregable

Se construye una vez y se reutiliza entre ambientes.

VALOR

Sin sorpresas entre ambientes.

La configuración va en el ambiente

Los datos de conexión y las direcciones vienen de afuera.

VALOR

Una sola imagen para todos los ambientes.

Describir lo que queremos

Archivos, revisiones de salud y un pipeline que comprueba.

VALOR

El sistema se repara solo y los errores aparecen antes.

Observabilidad: tres preguntas

Observar es poder responder qué pasa adentro del sistema, sin adivinar y sin entrar servidor por servidor.

¿Está vivo?

Cada servicio responde si está bien y si llega a su base.

¿Qué pasó?

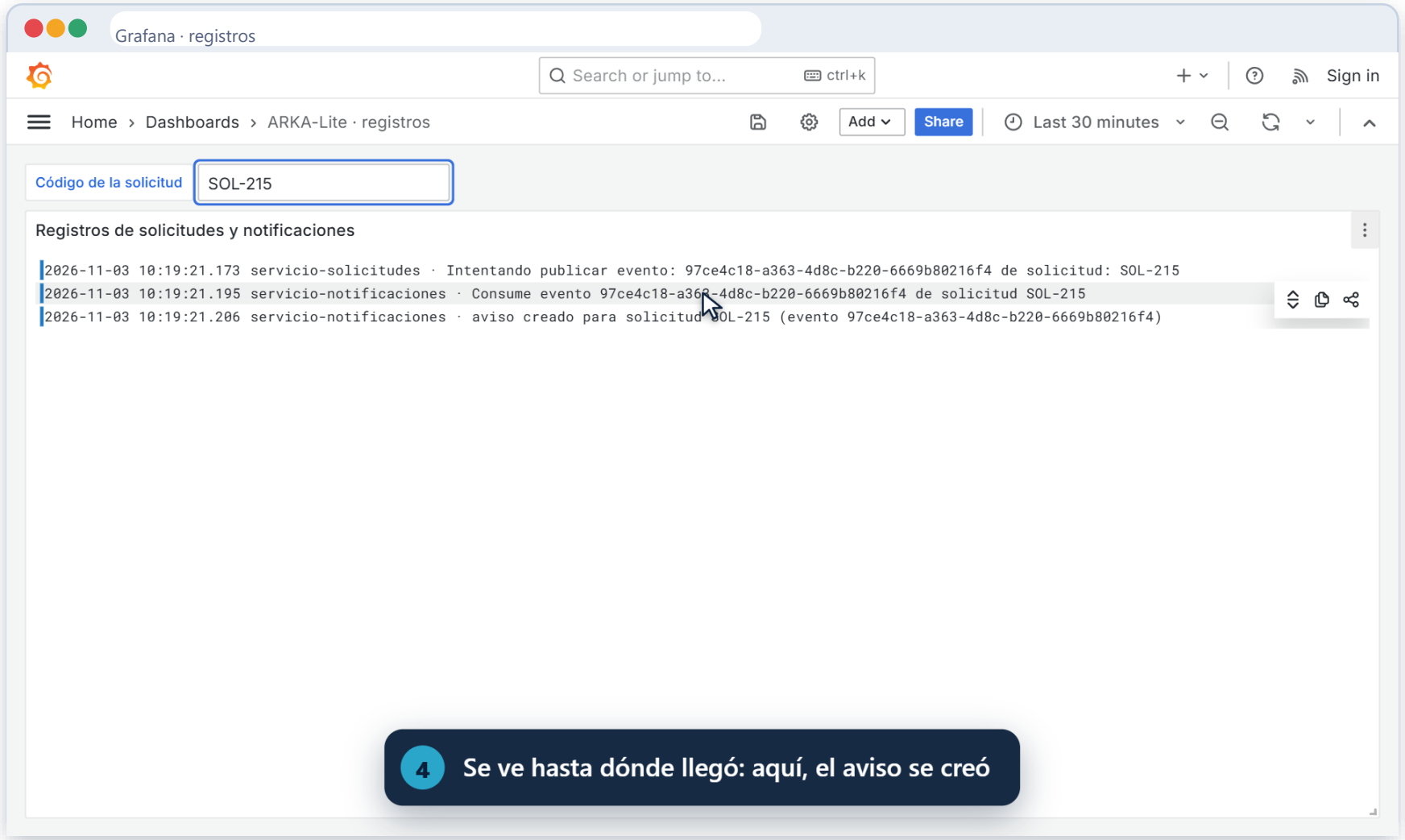
Los registros: la bitácora de lo que hizo cada servicio.

¿Por dónde pasó?

Un número de seguimiento que acompaña cada pedido.

Piénsenlo así: Como rastrear un paquete de courier: con el número de guía se ve por dónde pasó, cuándo y dónde se quedó.

Un aviso no llegó: ¿cómo lo encontramos?



QUÉ VAN A VER

1. Promtail recolecta y Loki guarda los registros de todos los servicios
2. En Grafana escribimos el código de la solicitud
3. Aparecen juntos los registros de solicitudes y de notificaciones
4. Se ve hasta dónde llegó: aquí, el aviso se creó

▶ Arranca solo · 0:27 · Alt+P pausa

Observabilidad: lo que aprendimos

LO QUE APRENDIMOS

1

lugar para buscar todos los registros

8

contenedores vigilados

Registros ordenados desde el primer día

Con un formato común, sin cambiar el código del negocio.

VALOR

Buscar por dato en lugar de leer texto.

Un número que siga cada pedido

Viaja en cada llamada y aparece en cada registro.

VALOR

Diagnosticar un fallo entre servicios en minutos.

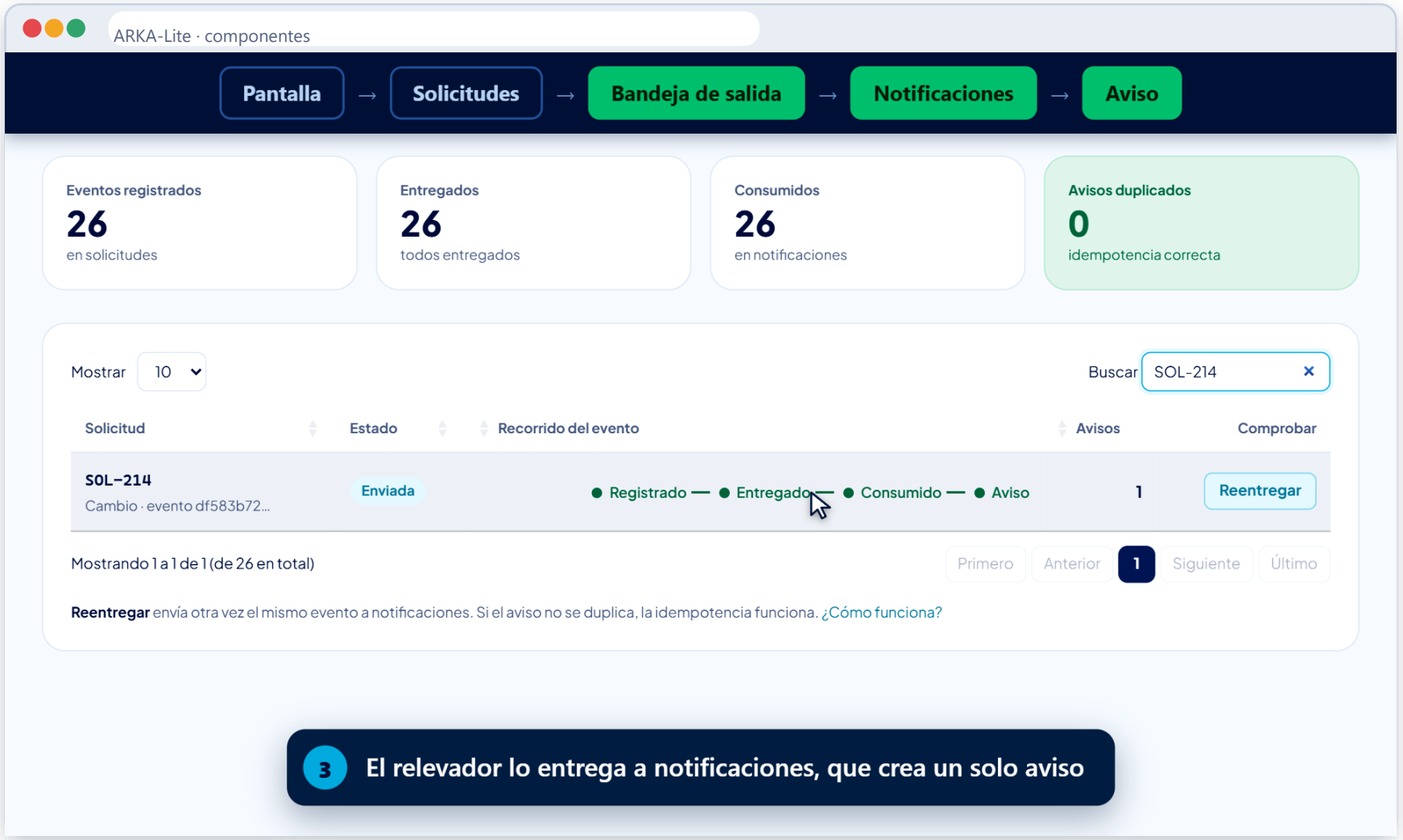
Encendido no es lo mismo que funcionando

Hay que revisar también el resultado.

VALOR

Medir lo que le importa al negocio.

En video: cómo interactúan los componentes



QUÉ VAN A VER

1. La pantalla le pide a solicitudes crearla: queda en BORRADOR
2. Al enviarla, solicitudes deja el evento en la bandeja de salida
3. El relevador lo entrega a notificaciones, que crea un solo aviso

▶ Arranca solo · 0:30 · Alt+P pausa

En video: si notificaciones se cae



QUÉ VAN A VER

1. Detenemos notificaciones a propósito
2. Enviamos una solicitud: responde bien, sin errores
3. El evento espera en la bandeja de salida y se reintenta cada 5 segundos
4. Levantamos notificaciones otra vez
5. Llega un solo aviso, con el mismo evento
6. Reentregamos el mismo evento a propósito: no se duplica

▶ Arranca solo · 1:07 · Alt+P pausa

Demostración: lo que aprendimos

LO QUE APRENDIMOS

0

eventos perdidos durante la caída

1

aviso por evento, aunque se reentregue

Diseñar para la falla

Cada falla probada tiene una reacción y un mensaje.

VALOR

Menos incidentes y menos llamadas a soporte.

Hacer visible lo asíncrono

Pendientes y entregados en una pantalla.

VALOR

Explicar el sistema sin abrir el código.

Verlo funcionar explica más

Provocar la falla y ver la reacción.

VALOR

Material de capacitación para quien llegue al equipo.

Resultados para el negocio

Lo que el sistema logra, en el idioma del negocio, y cómo lo comprobamos.

RESULTADO	QUÉ SIGNIFICA PARA EL NEGOCIO	CÓMO LO COMPROBAMOS
Entregas sin pérdidas ni duplicados	Lo pendiente queda guardado, y reenviarlo no crea otro aviso	Tumbamos un servicio y reentregamos el mismo evento
El trámite no se detiene	Se puede enviar aunque el servicio de avisos esté caído	Enviamos con el servicio de avisos apagado
Las reglas se cumplen	No se envía dos veces ni se aprueba un borrador; el sistema explica por qué	Una prueba automática por cada regla
Reemplaza lo que falla	Una copia que falla se reemplaza, y lo pendiente sigue guardado	Kubernetes reemplaza el servicio caído
Cambios con menos riesgo	Un error se detecta en minutos, antes de llegar a los usuarios	Cada cambio pasa por pruebas automáticas

Lo que sostiene los resultados

Detrás de cada resultado hay una decisión técnica concreta.

PILAR	QUÉ CONSTRUIMOS	QUÉ APORTA
Datos consistentes	Transacción, bandeja de salida e idempotencia	Eventos guardados y sin avisos duplicados ante caídas
Configuración separada	Datos de conexión y claves fuera del código	Cambiar de servidor o de clave sin tocar el sistema
Reproducibilidad	Docker, Compose y Kubernetes	1 comando y el mismo sistema en todo ambiente
Calidad	Pruebas automáticas y pipeline en cada cambio	Errores detectados en minutos, antes de integrar
Trazabilidad	Registros en un solo lugar y número de seguimiento	Saber qué pasó y qué versión corre, en minutos

Lo que nos llevamos

LO QUE APRENDIMOS

0

eventos perdidos si un servicio se cae

1

comando para levantar todo

Diseñar para la falla

Guardar primero, reintentar e ignorar duplicados.

VALOR

Sistemas que se recuperan solos.

Automatizar las revisiones

Pruebas, pipeline e imágenes.

VALOR

Cambiar rápido sin miedo a romper.

Hacer visible el sistema

Registros, seguimiento y salud.

VALOR

Diagnosticar y explicar en minutos.

A ARKA-Lite

Gracias

Equipo arka-litos

Proyecto final · Upskilling Backend Java



Reparto de la presentación

Lámina de apoyo para el equipo: se quita antes de presentar

Persona	Área	Láminas	Qué presenta
Luis Cardoze	Desarrollo	1–5	Apertura · Fundamentos
Ricardo Fábrega	Desarrollo	6–9	Arquitectura
Marlene Bonilla	Base de datos	10–12 y 17–18	Base de datos
Jossett Franceschi	Desarrollo	13–16 y 29–37	Apoyo en base de datos: la misma regla, el ORM y la conexión · DevOps: CI/CD
Abdiel Barria	Desarrollo	19–23 y 46–52	Interconexión de servicios · Demostración en video · Resultados, valor y cierre
Kevin	Infraestructura	24–28	DevOps: qué es, Docker y Compose
Moises	Infraestructura	38–45	DevOps: Kubernetes, y la observabilidad